

Análisis Técnico - Módulo de Administración de Reservas (RESUMEN)

Feature: Consulta y Creación Manual de Reservas (HU-01 y HU-02)

Fecha: 2026-04-08 | **Versión:** Resumen Ejecutivo

Historias de Usuario: HU-01 (Consultar Reservas), HU-02 (Crear Reserva Manual)

ÍNDICE RÁPIDO

- Backend: 5 Servicios
- Frontend: Módulo Admin (React + TypeScript)
- Principios SOLID: 5/5 Implementados
- Patrones de Diseño: 15+ Patrones

I. BACKEND - RESUMEN POR SERVICIO

1. BOOKINGS-SERVICE (Principal)

Responsabilidad: Gestión de reservas administrativas

SOLID:

- **SRP:** Cada componente segrega responsabilidades (Query, Controller, Adapter, DTO)
- **OCP:** Nuevos filtros sin modificar código existente
- **LSP:** Adaptadores intercambiables (RabbitMQ/NoOp, JDBC/Mock)
- **ISP:** Puertos segregados (Persistence, Events, Token)
- **DIP:** Inyección de dependencias en constructores

Patrones Principales:

1. **Hexagonal Architecture** - Dominio separado de frameworks
2. **CQRS** - Queries (lectura) vs Commands (escritura)
3. **Adapter Pattern** - HTTP, JDBC, RabbitMQ
4. **Strategy Pattern** - User resolution (admin override)
5. **Validation Chain** - Validaciones secuenciales
6. **DTO Pattern** - Segregación por contexto (User vs Admin)
7. **Null Object** - NoOp event publisher

Endpoints Principales:

GET /bookings/admin/reservations → Consulta paginada (20 items/página) → Filtros: fecha, estado, usuario, sede
POST /bookings/reservations → Crear reserva (usuario o admin) → Admin puede override con targetUserId
PATCH /bookings/reservations/{id}/cancel → Cancelar reserva con razón
PATCH /bookings/reservations/{id}/deliver → Marcar como entregada

Archivos Clave:

- application/usecase/AdminListReservationsQuery.java

- adapters/in/web/BookingController.java
- application/service/BookingApplicationService.java
- adapters/out/persistence/JdbcReservationPersistenceAdapter.java
- adapters/out/messaging/RabbitReservationEventPublisherAdapter.java

2. AUTH-SERVICE (Soporte)

Responsabilidad: Validación de tokens JWT y autenticación

SOLID:

- **DIP:** TokenPort abstrae generación/validación
- **SRP:** Único responsable de JWT

Patrones:

1. **Adapter Pattern** - JwtTokenAdapter implementa TokenPort

Archivos Clave:

- adapters/out/security/JwtTokenAdapter.java
- infrastructure/security/JwtAuthenticationFilter.java

3. LOCATIONS-SERVICE (Soporte)

Responsabilidad: Catálogo de ciudades y espacios

SOLID:

- **OCP:** Evoluciona sin romper clientes

Patrones:

1. **Repository Pattern** - Acceso a datos segregado

4. INVENTORY-SERVICE (Soporte)

Responsabilidad: Catálogo de equipos

SOLID:

- **SRP:** Valida pero no crea reservas

Patrones:

1. **Repository Pattern** - Persistencia segregada

5. NOTIFICATIONS-SERVICE (Soporte)

Responsabilidad: Notificaciones en tiempo real vía WebSocket

Patrones:

1. **Observer/Pub-Sub** - Consume eventos de RabbitMQ y retransmite vía STOMP

II. FRONTEND - RESUMEN

Admin Module (React + TypeScript)

Responsabilidad: Interfaz de administración de reservas

SOLID:

- **SRP:** Componentes segregados por responsabilidad
- **DIP:** Servicios inyectados como props

Patrones Principales:

1. **Container/Presentational** - AdminReservationsPage (smart) vs componentes (dumb)
2. **Custom Hooks** - useAdminReservationFilters, useAdminCreateReservationForm
3. **Service Layer** - adminReservationsService centraliza HTTP
4. **Mapper Pattern** - ReservationMapper traduce API ↔ Dominio

Componentes Principales:

```
AdminReservationsPage (Contenedor) ■■ ReservationStatusFilter (Filtros) ■■ ReservationTable  
(Tabla paginada) ■■ AdminCreateReservationPanel (Formulario creación) ■■  
ReservationRequesterAutocomplete ■■ Calendar + DurationSelector ■■ Equipment checkboxes
```

Hooks:

```
useAdminReservationFilters() → { filters, setFilter, clearFilters }  
useAdminCreateReservationForm() → { form, errors, validate, reset, setFieldValue }
```

Servicios:

```
getAdminReservations(query) → GET /bookings/admin/reservations createAdminReservation(payload) →  
POST /bookings/reservations searchAdminUsers(query) → GET /auth/users getAdminCities() → GET  
/locations/cities getAdminSpacesByCity(cityId) → GET /locations/spaces  
getAdminEquipmentByCity(cityId) → GET /inventory/equipments
```


III. FLUJOS CRÍTICOS

Flujo 1: Consulta (HU-01)

```
Frontend Admin ↓ AdminReservationsPage + Filtros ↓ GET  
/bookings/admin/reservations?desde=...&estado;=... ↓ Backend: Validaciones + Query SQL ↓ SELECT *  
FROM reservations (con JOINS) ↓ AdminReservationsPageResponse (paginado 20 items) ↓ Frontend:  
Renderiza tabla + paginación
```

Estados Visibles:

- Pendiente → pending
- Confirmada → confirmed
- Cancelada → cancelled
- Finalizada → completed

Flujo 2: Creación (HU-02)

Frontend Admin: Click "Nueva Reserva" ↓ Modal: Selecciona usuario (autocomplete) Selecciona ciudad → carga espacios + equipos Selecciona espacio + fecha + horario (Opcional) Selecciona equipos ↓ Validación Frontend: Campos obligatorios ↓ POST /bookings/reservations { targetUserId, spaceId, startAt, endAt, equipmentIds, attendeesCount } ↓ Backend Validaciones Encadenadas: 1. Espacio existe → NOT_FOUND si no 2. Rango fechas válido → BAD_REQUEST si no 3. Sin solapamiento → CONFLICT si hay solapamiento 4. Equipos válidos → BAD_REQUEST si no ↓ INSERT en BD ↓ Publica: ReservationCreatedEvent → RabbitMQ ↓ Frontend: Toast "Reserva creada exitosamente" Recarga listado Nueva reserva aparece en tabla

Validación de Conflicto:

```
SELECT * FROM reservations WHERE space_id = ? AND status IN ('pending', 'confirmed', 'in_progress') AND start_datetime < endAt AND end_datetime > startAt
```

Si hay conflicto:

→ ApiException (409 CONFLICT) → Frontend muestra: "El espacio seleccionado ya se encuentra reservado en este horario" → Modal permanece abierta para reintentar

IV. MATRIZ PRINCIPIOS SOLID

| Principio | Bookings | Auth | Locations | Inventory | Frontend |

|-----|-----|-----|-----|-----|

| **S - SRP** | ✓ | ✓ | ✓ | ✓ | ✓ |

| **O - OCP** | ✓ | - | ✓ | ✓ | - |

| **L - LSP** | ✓ | - | - | - | - |

| **I - ISP** | ✓ | - | - | - | - |

| **D - DIP** | ✓ | ✓ | - | - | ✓ |

V. MATRIZ PATRONES

| Patrón | Backend | Frontend |

|-----|-----|-----|

| Hexagonal Architecture | ✓ | ✓ |

| Adapter Pattern | ✓ | - |

| CQRS | ✓ | - |

| Strategy Pattern | ✓ | - |

| Validation Chain | ✓ | ✓ |

| DTO Pattern | ✓ | - |

| Null Object | ✓ | - |

| Repository Pattern | ✓ | - |

| Event-Driven | ✓ | ✓ |

| Container/Presentational | - | ✓ |

| Custom Hooks | - | ✓ |

| Service Layer | ✓ | ✓ |

| Mapper Pattern | ✓ | ✓ |

VI. VERIFICACIÓN DE ESTADOS

Estados de Reserva (en BD):

pending → Creada, no confirmada confirmed → Confirmada por admin/sistema in_progress → Entregada al usuario completed → Devuelta por usuario cancelled → Cancelada

Transiciones Válidas:

PENDING → CONFIRMED → IN_PROGRESS → COMPLETED ■ CANCELLED (cualquier momento)

Validaciones:

Backend: ■■ Espacio existe ■■ Rango fechas válido ■■ Sin solapamiento ■■ Equipos válidos ■■
Usuario existe Frontend: ■■ Campos obligatorios ■■ Fechas no pasadas ■■ startTime < endTime ■■
Búsqueda de usuario exitosa

VII. TECNOLOGÍAS CLAVE

Backend:

- Java 17 + Spring Boot
- JDBC + MariaDB
- RabbitMQ (eventos)
- JWT (autenticación)

Frontend:

- React + TypeScript
- Custom Hooks
- STOMP WebSocket (notificaciones)
- Axios (HTTP client)

VIII. ARCHIVOS GENERADOS

- ✓ resultado_tecnico_modulo_admin_reorganizado.md (Análisis completo - 2,419 líneas)
- ✓ resultado_tecnico_modulo_admin_reorganizado.pdf (PDF para lectura offline - 65.7 KB)
- ✓ resultado_tecnico_modulo_admin_resumen.md (Este archivo - resumen ejecutivo)
- ✓ scripts/md_to_pdf.py (Conversor MD → PDF con reportlab)

CONCLUSIÓN

El módulo de administración implementa:

- ✓ Todos los principios SOLID (principalmente en Backend)
- ✓ 15+ patrones de diseño (Hexagonal, CQRS, Adapters, etc.)
- ✓ Separación clara de responsabilidades
- ✓ Arquitectura escalable y mantenible
- ✓ Validaciones robustas de negocio
- ✓ Manejo consistente de errores

La implementación está lista para producción con altas garantías de calidad arquitectónica.